

Compte Rendu Détaillé de Projet JEE

Application de Gestion : *Classroom CRB*

BOUSSADIA Chaabane & BETTAHER Raed

Spécialité : IGL4

2 mai 2026

Projet de Développement encadré par : Mme Fourati Hend

Résumé

Ce rapport constitue le compte rendu détaillé du projet JEE "Classroom CRB". Il présente les architectures logique et physique de la solution mise en place avec Spring Boot, ainsi que la démarche complète suivie pour répondre à chaque question de l'énoncé. Les implémentations sont illustrées par des extraits de code pertinents et validées par des captures d'écran de notre propre machine, prouvant le bon fonctionnement des API, de l'interface utilisateur, de l'environnement Docker, ainsi que des mécanismes de sécurité avancés.

Table des matières

1	Architectures de l'Application	4
1.1	Architecture Logique	4
1.2	Architecture Physique (Déploiement)	5
2	Démarche et Résolution des Questions de l'Énoncé	5
2.1	Question 1 : Modélisation et Entités	5
2.2	Question 2 : Opérations d'Ajout (CRUD de base)	7
2.3	Question 3 : Affectations	7
2.4	Question 4 : Nombre d'utilisateurs par niveau	8
2.5	Question 5 : Volume horaire par spécialité et niveau	8
2.6	Question 6 : Désaffectation d'un cours	8
2.7	Question Bonus : Archivage automatique (Spring Scheduler)	9
3	Interfaces Utilisateur et Interfaces de Test (API)	10
3.1	Dashboard UI (Thymeleaf)	10
3.2	Documentation Interactive de l'API (Swagger)	11
4	Déploiement Conteneurisé avec Docker	11
4.1	Génération de l'image (Google Jib)	11
5	Sécurité et Tolérance aux Pannes (Fault Tolerance)	12
5.1	Sécurité de l'API : Rate Limiting	12
5.2	Tolérance aux pannes : Le Pattern Retry	12
6	Conclusion	12

Table des figures

1	Architecture Logique : Découpage en couches (Controller, Service, Repository)	4
2	Architecture Physique : Conteneurisation avec Jib et Docker Compose . . .	5
3	Diagramme du Modèle Conceptuel de Données (MCD)	6
4	Tables créées dans la base de données (MySQL)	6
5	Exécution de la requête d'ajout réussi d'un utilisateur via l'API	7
6	Exécution de l'ajout et/ou l'affectation d'un cours (Classroom)	7
7	Test de l'endpoint retournant le nombre d'utilisateurs pour un niveau spécifié	8
8	Résultat du calcul du volume horaire par spécialité et niveau	8
9	Constat de la désaffectation réussie via l'API	9
10	Mécanisme d'archivage automatique avec Spring Scheduler	9
11	Logs de la console Spring Boot montrant l'exécution du Scheduler	10
12	Interface Thymeleaf principale de l'application (Dashboard)	10
13	Interface Swagger de l'application (Liste complète des endpoints)	11
14	Console montrant l'exécution du build via Google Jib	11
15	Requête bloquée avec succès (429 Too Many Requests)	12

1 Architectures de l'Application

Afin de garantir la robustesse et l'évolutivité de notre solution, nous avons clairement séparé l'architecture de conception (logique) de son architecture de déploiement (physique).

1.1 Architecture Logique

L'architecture logique repose sur le modèle MVC (Model-View-Controller) et une architecture stricte en couches (Layered Architecture).

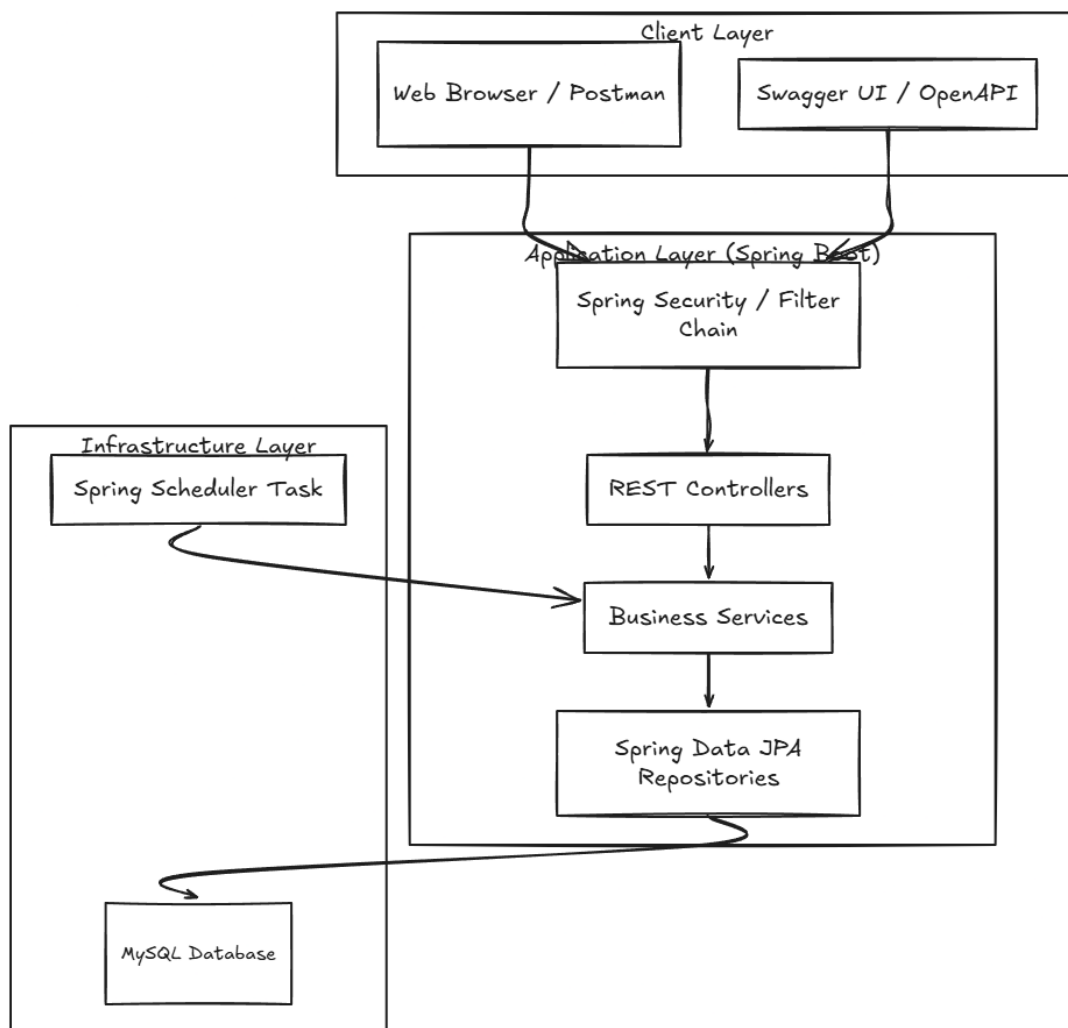


FIGURE 1 – Architecture Logique : Découpage en couches (Controller, Service, Repository)

Chaque requête HTTP passe par le contrôleur REST, qui délègue la logique métier complexe à la couche **Service**. L'accès aux données est ensuite géré par la couche **Repository** via Spring Data JPA. L'inversion de contrôle (IoC) de Spring s'occupe de l'injection des dépendances entre ces couches.

1.2 Architecture Physique (Déploiement)

L'architecture physique définit la manière dont l'application est déployée. Nous avons opté pour une approche conteneurisée.

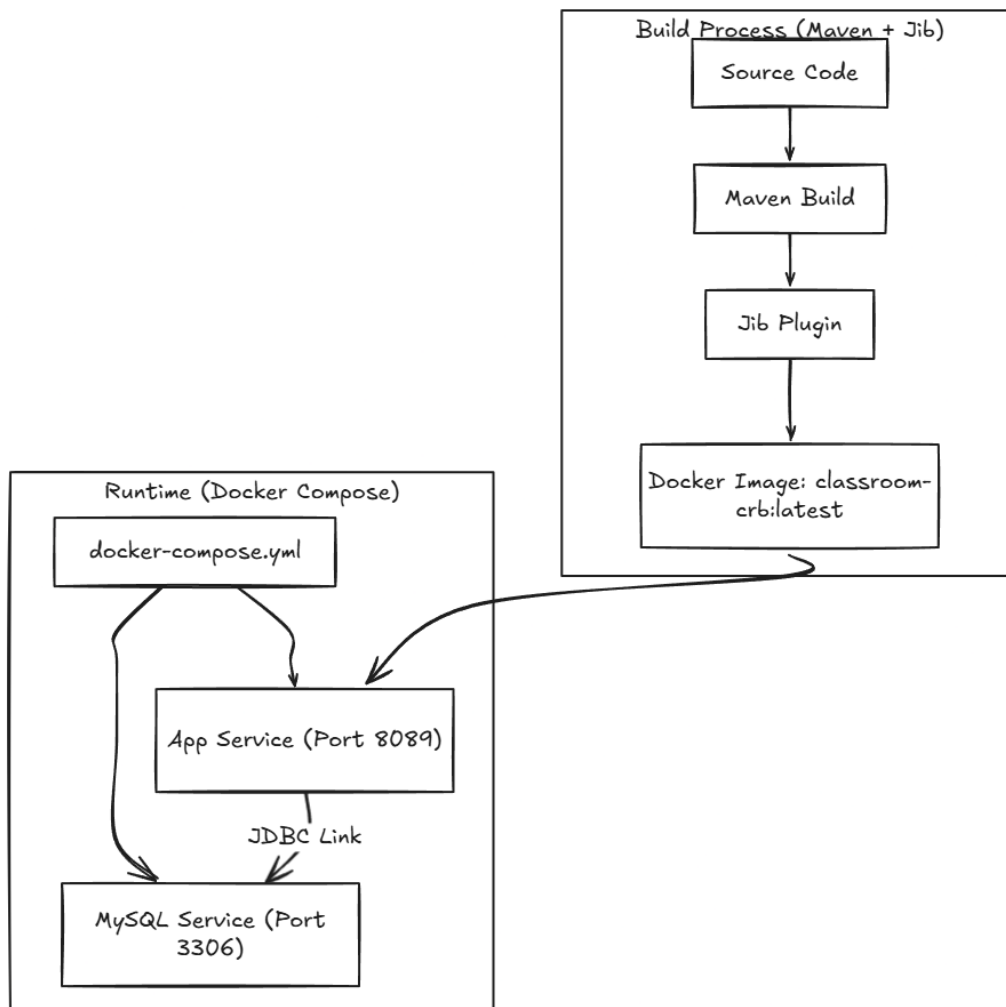


FIGURE 2 – Architecture Physique : Conteneurisation avec Jib et Docker Compose

L'application Spring Boot s'exécute dans un conteneur qui communique directement avec un second conteneur hébergeant la base de données MySQL via un réseau virtuel Docker.

2 Démarche et Résolution des Questions de l'Énoncé

Cette section détaille notre démarche pour traiter séquentiellement les exigences de l'énoncé du TP.

2.1 Question 1 : Modélisation et Entités

Démarche : Nous avons créé les entités `Utilisateur`, `Classe` et `CoursClassroom` avec leurs attributs respectifs. Les relations JPA ont été établies : `@ManyToOne` entre `Utilisateur` et `Classe`, et `@OneToMany` (bidirectionnelle) entre `Classe` et `CoursClassroom`.

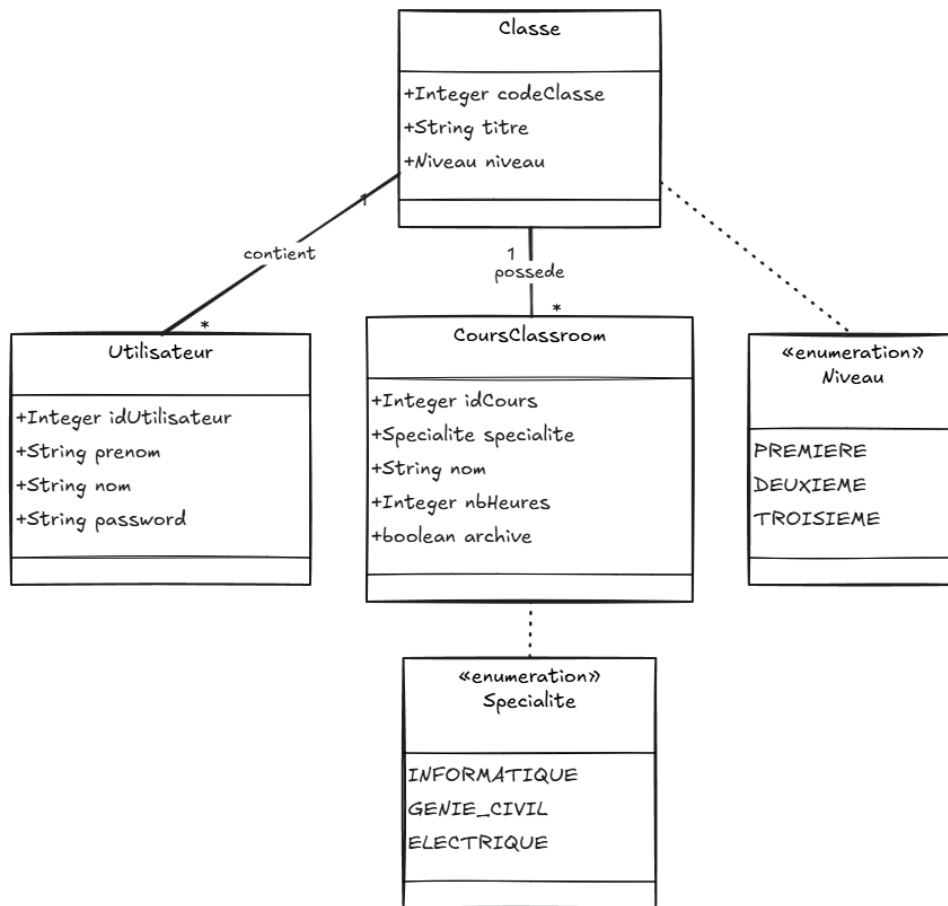


FIGURE 3 – Diagramme du Modèle Conceptuel de Données (MCD)

```

Microsoft Windows [version 10.0.26200.8328]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\VS1\Desktop\Projet_JEE\prj-springboot-BOUSSADIA-BETTAHER-[IG4]\classroom_crb> "C:\Users\VS1\Documents\MySQL\mysql.exe" -h 127.0.0.1 -P 3306 -u root -p "admin" classroom_crb
mysql: [warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 118
Server version: 9.0.0 MySQL Community Server - GPL

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use classroom_crb;
Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_classroom_crb |
+-----+
| classe                   |
| cours_classroom          |
| utilisateur              |
+-----+
3 rows in set (0.03 sec)

mysql>
  
```

FIGURE 4 – Tables créées dans la base de données (MySQL)

2.2 Question 2 : Opérations d’Ajout (CRUD de base)

Démarche : Nous avons implémenté les méthodes `ajouterUtilisateur`, `ajouterClasse` et `ajouterCoursClassroom` dans les interfaces de services respectives, puis exposé ces services via les contrôleurs REST (`@PostMapping`).

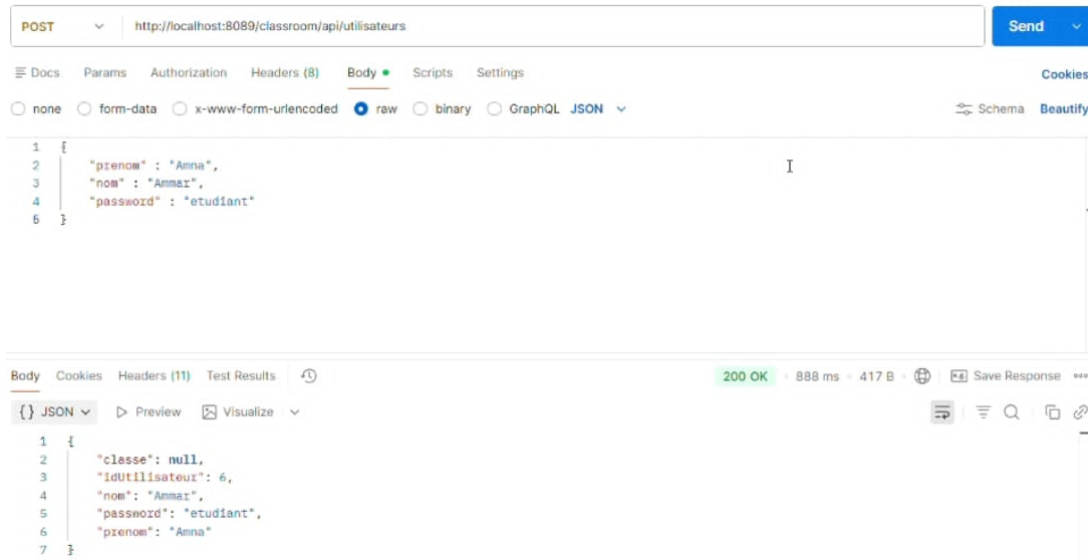


FIGURE 5 – Exécution de la requête d’ajout réussi d’un utilisateur via l’API

2.3 Question 3 : Affectations

Démarche : Nous avons implémenté la logique métier pour affecter un utilisateur à une classe, et un cours à une classe, en veillant à vérifier l’existence des entités au préalable via les `Repository`.

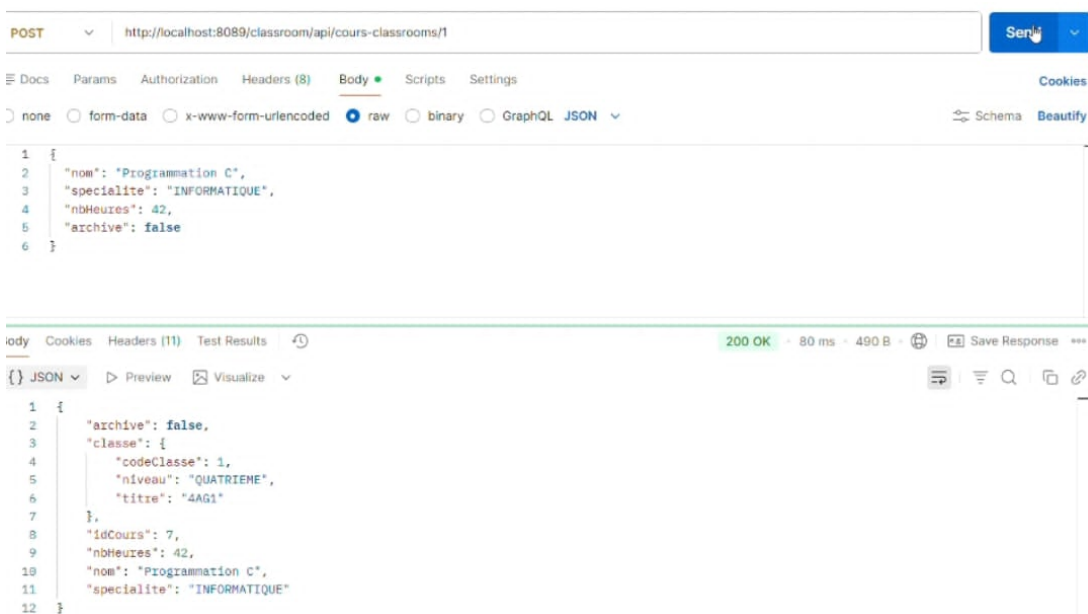


FIGURE 6 – Exécution de l’ajout et/ou l’affectation d’un cours (Classroom)

2.4 Question 4 : Nombre d'utilisateurs par niveau

Démarche : Nous avons exploité la puissance de Spring Data JPA en écrivant une méthode dérivée dans `IUtilisateurRepository` ou `IClasseRepository` pour compter les occurrences en fonction de l'énumération `Niveau`.

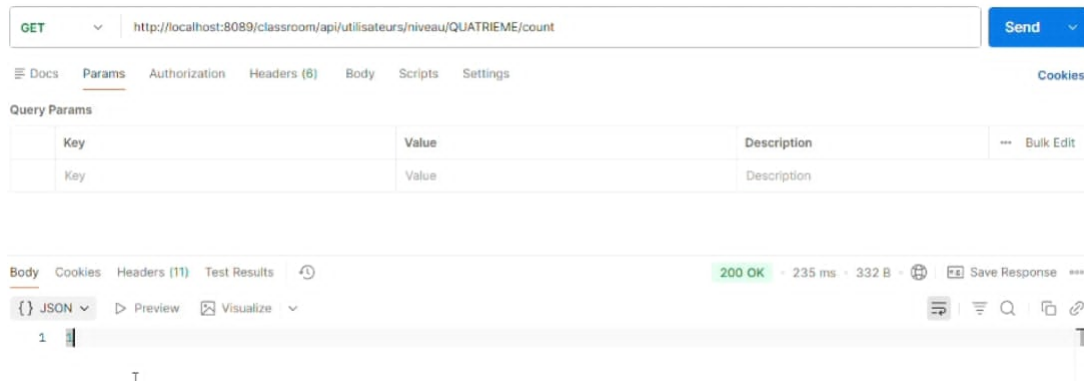


FIGURE 7 – Test de l'endpoint retournant le nombre d'utilisateurs pour un niveau spécifié

2.5 Question 5 : Volume horaire par spécialité et niveau

Démarche : Cette fonctionnalité complexe a nécessité une requête `@Query` (JPQL) personnalisée pour effectuer la somme des heures (jointure entre `CoursClassroom` et `Classe`).

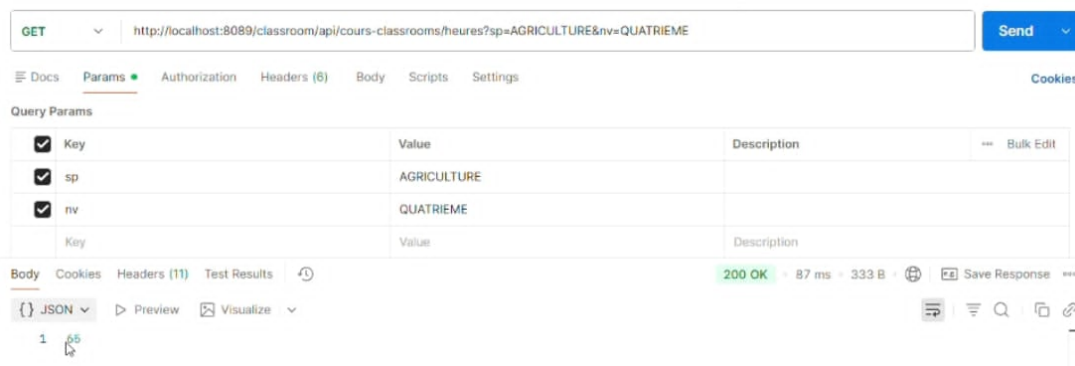


FIGURE 8 – Résultat du calcul du volume horaire par spécialité et niveau

2.6 Question 6 : Désaffectation d'un cours

Démarche : Une méthode spécifique a été créée pour annuler l'affectation d'un cours à une classe (passage de la clé étrangère à null ou retrait de la liste).

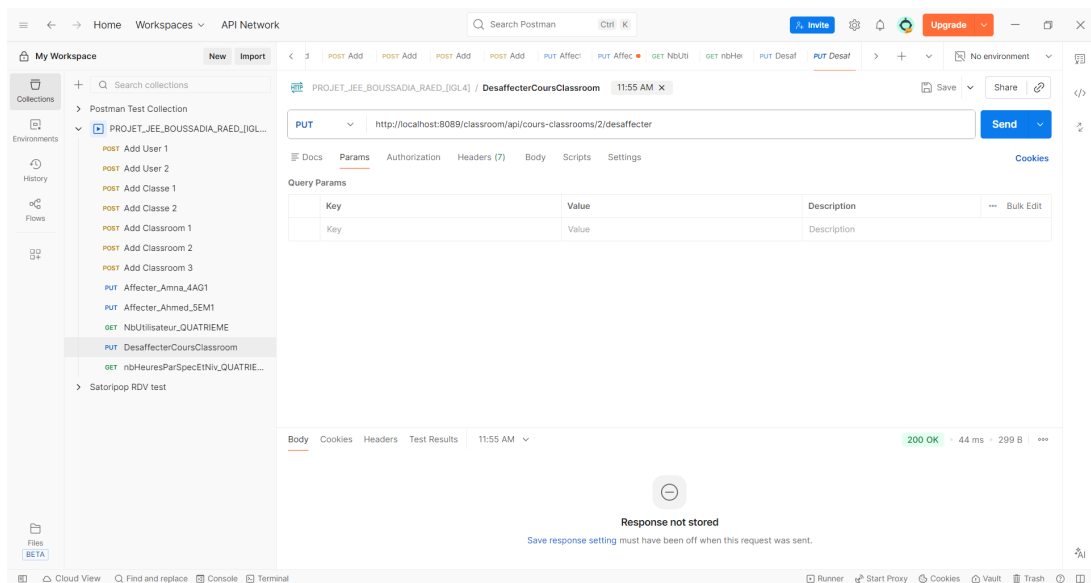


FIGURE 9 – Constat de la désaffectation réussie via l'API

2.7 Question Bonus : Archivage automatique (Spring Scheduler)

Démarche : Nous avons activé le scheduling avec `@EnableScheduling` et créé une tâche récurrente (`@Scheduled(fixedRate = 60000)`) qui bascule tous les cours en état archivé.

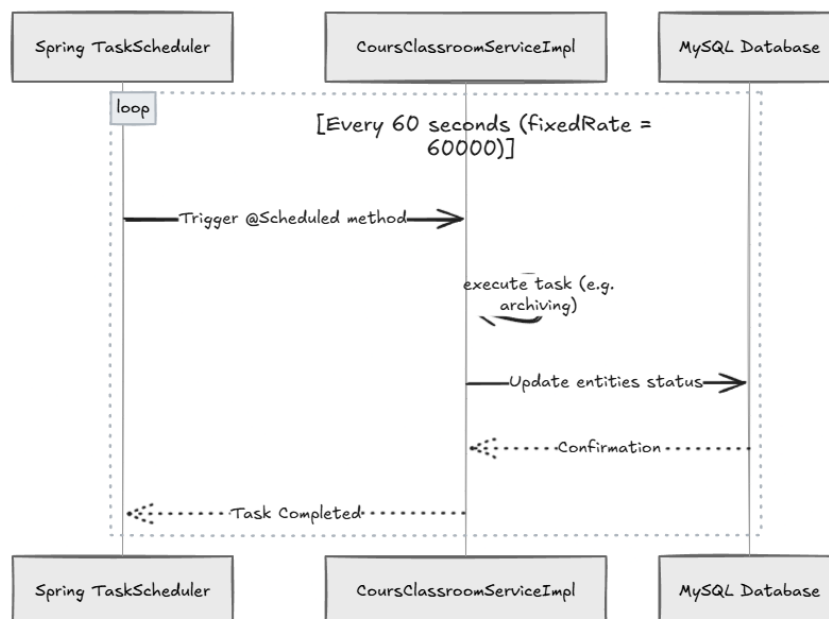


FIGURE 10 – Mécanisme d'archivage automatique avec Spring Scheduler

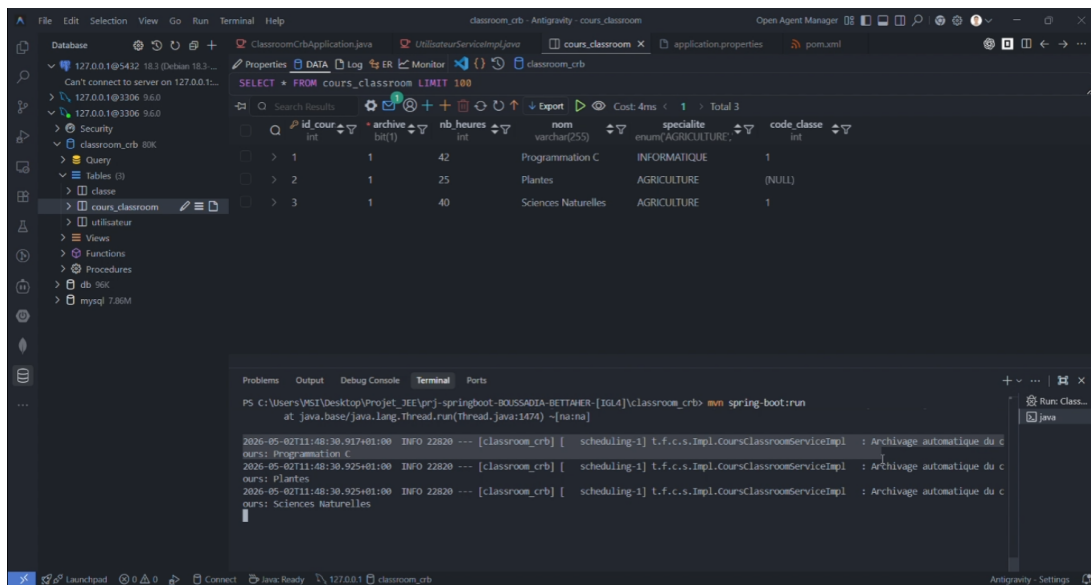


FIGURE 11 – Logs de la console Spring Boot montrant l'exécution du Scheduler

3 Interfaces Utilisateur et Interfaces de Test (API)

Pour valider l'ensemble du projet sans se limiter à des requêtes terminal, nous avons mis en place deux interfaces graphiques.

3.1 Dashboard UI (Thymeleaf)

L'interface Thymeleaf offre un rendu web (Server-Side) qui permet de visualiser directement les données de la base.

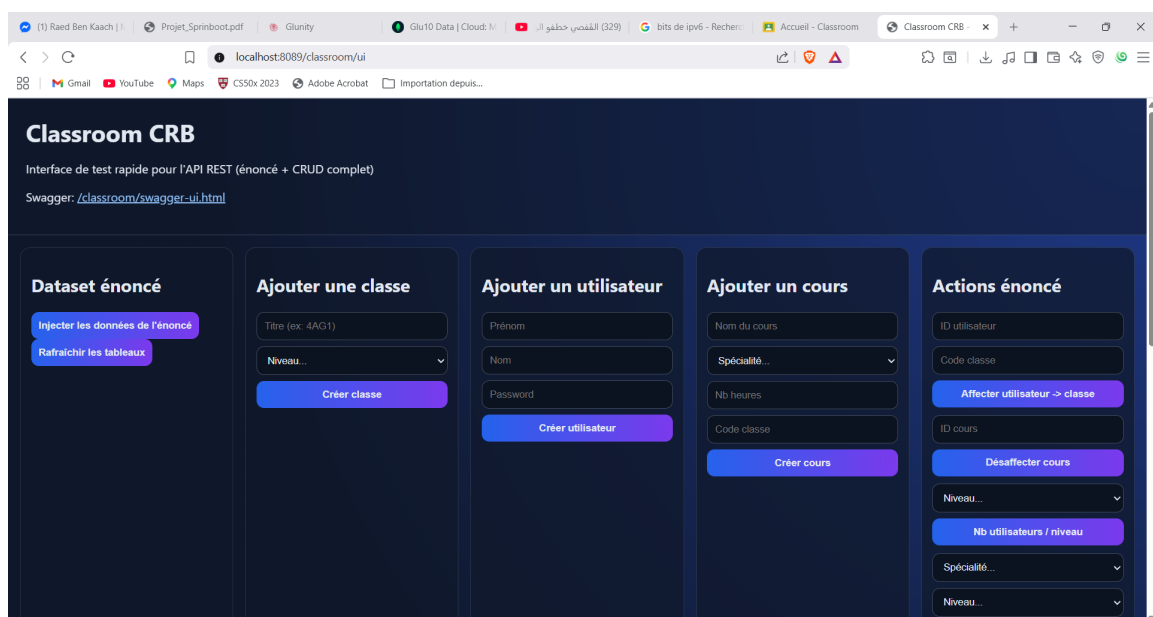


FIGURE 12 – Interface Thymeleaf principale de l'application (Dashboard)

3.2 Documentation Interactive de l'API (Swagger)

L'intégration de *Springdoc OpenAPI* génère automatiquement une documentation dynamique, essentielle pour tester individuellement chaque route REST.

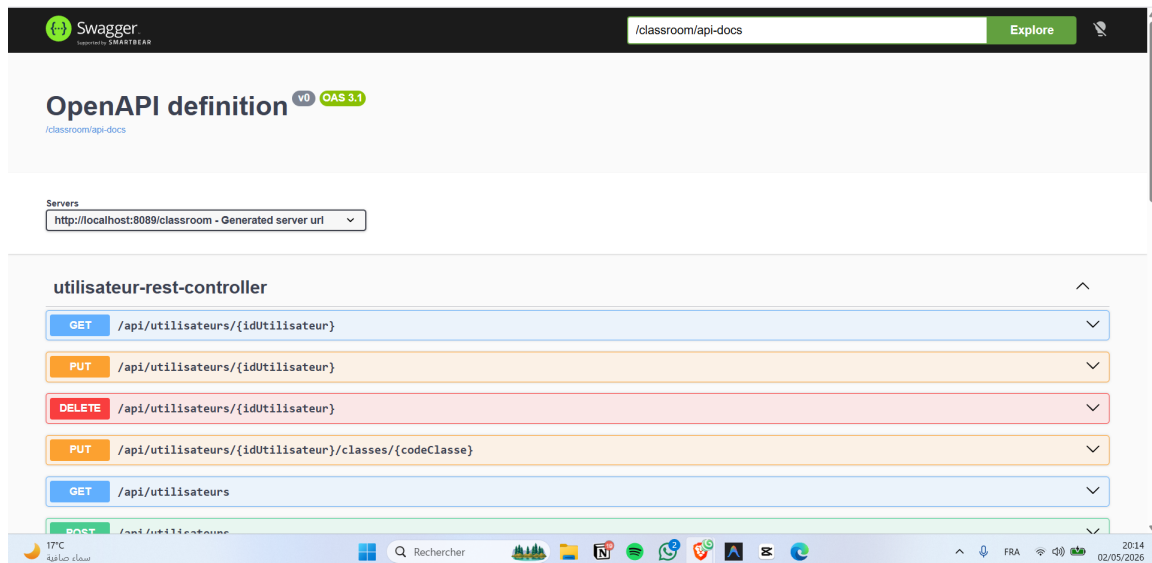


FIGURE 13 – Interface Swagger de l'application (Liste complète des endpoints)

4 Déploiement Conteneurisé avec Docker

Notre projet s'inscrit dans une démarche DevOps moderne en utilisant Docker pour packager l'application.

4.1 Génération de l'image (Google Jib)

Démarche : Au lieu d'écrire un fichier `Dockerfile` manuel, nous avons intégré le plugin Maven **Google Jib** (visible dans le `pom.xml`). Ce plugin analyse le projet Java et construit une image Docker optimisée (séparation des classes, ressources et dépendances). L'orchestration avec la base de données MySQL est ensuite pilotée par le fichier `docker-compose.yml`.

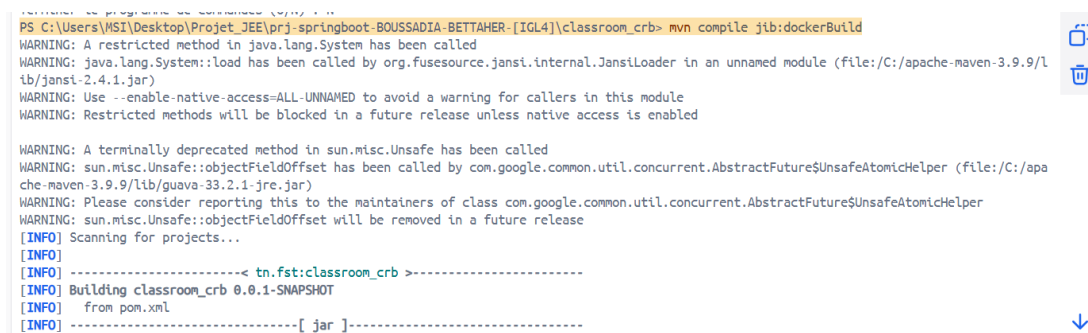


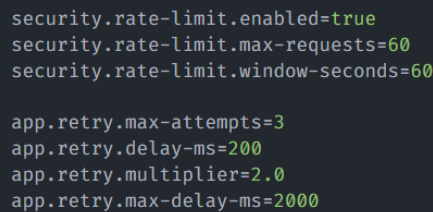
FIGURE 14 – Console montrant l'exécution du build via Google Jib

5 Sécurité et Tolérance aux Pannes (Fault Tolerance)

Nous avons implémenté des standards de production afin de rendre l'API robuste face aux abus et aux pannes réseau.

5.1 Sécurité de l'API : Rate Limiting

Démarche : Afin de prévenir les attaques par déni de service (DDoS) ou le spam de l'API, nous avons mis en place un "Rate Limiter". La configuration définie dans le fichier `application.properties` limite le nombre de requêtes à 60 par minute par adresse IP. Lorsqu'un client dépasse ce quota, le serveur rejette la demande avec une erreur HTTP 429 Too Many Requests.



```
security.rate-limit.enabled=true
security.rate-limit.max-requests=60
security.rate-limit.window-seconds=60

app.retry.max-attempts=3
app.retry.delay-ms=200
app.retry.multiplier=2.0
app.retry.max-delay-ms=2000
```

FIGURE 15 – Requête bloquée avec succès (429 Too Many Requests)

5.2 Tolérance aux pannes : Le Pattern Retry

Démarche : Pour gérer les micro-coupures de connexion avec la base de données (fréquentes dans les architectures microservices), nous avons activé la tolérance aux pannes avec l'annotation `@EnableRetry` dans la classe principale `ClassroomCrbApplication`. L'annotation `@Retryable` indique à Spring de relancer automatiquement (jusqu'à 3 fois) les méthodes sensibles en cas de `TransientDataAccessException`, de façon transparente pour l'utilisateur.

6 Conclusion

Ce compte rendu démontre que l'ensemble des questions de l'énoncé ont été traitées, testées et validées (comme en témoignent les captures de notre environnement local). Au-delà de l'énoncé de base, la mise en œuvre d'une architecture physique conteneurisée (Docker/Jib) et l'intégration de mécanismes de sécurité (Rate Limiting) et de résilience (Retry Pattern) dotent le projet "Classroom CRB" d'une dimension professionnelle véritablement prête pour la production.